Architecture Technique de l'Écosystème eSchool / MonEcole

Guide Technique Approfondi

Pour Ingénieurs et Développeurs

Django • MySQL/MariaDB • ReportLab • Gunicorn

Version 1.0 — Mars 2026

evanet08 — github.com/evanet08

Table des matières

1 Vue d'Ensemble Technique **2**

1.1 Stack Technologique 2

1.2 Repositories Git 2

1.3 Arborescence des Projets 2

2 Architecture Multi-Base de Données **4**

2.1 Vue d'Ensemble du Schéma Distribué 4

2.2 Le DatabaseRouter 4

2.3 Pattern managed=False 5

2.4 VIEWs MySQL 5

2.4.1 VIEWs de Référence Simples 5

2.4.2 VIEW Cross-DB avec JOIN : cours_par_classe 6

2.4.3 VIEW Universal Context : annee_trimestre 6

3 Schéma de Données — eSchoolStructure **8**

3.1 Diagramme Entité-Relation Complet 8

3.2 Modèles Détaillés 9

3.2.1 Pays — Point d'Entrée 9

3.2.2 Cycle — Niveaux d'Enseignement 9

3.2.3 Cours — Catalogue des Matières 9

3.2.4 AdminUser — Hiérarchie de Sécurité 10

4 Schéma de Données — MonEcole **11**

4.1 Vue d'Ensemble 11

4.2 Modèle Élève — Fiche Complète 11

4.3 Circuit de la Note 12

5 Liaison Hub ↔ Spoke **13**

5.1 Chaîne de Résolution du Tenant 13

6 Système d'Authentification **14**

6.1 Machine à États 14
6.2 Endpoints d'Authentification 14
6.3 Décorateurs de Sécurité 14

7 Patterns de Développement 16
7.1 Pattern Upsert JSON 16
7.2 Pattern Checkbox Activation 17

8 Déploiement et DevOps 18
8.1 Workflow de Déploiement 18
8.2 Vérification Post-Déploiement 18

9 Guide du Développeur 20
9.1 Comment Ajouter un Nouveau Modèle 20
9.2 Comment Ajouter un Endpoint API 20
9.3 Comment Créer une Migration Personnalisée 20

A Variables d'Environnement 22

B Liste Complète des VIEWs MySQL 23

Chapitre 1

Vue d'Ensemble Technique

1.1 Stack Technologique

Client — Navigateur Web
HTML5 • CSS3 (Variables, Glassmorphism) • JavaScript (Vanilla, AJAX/Fetch)
Bootstrap 5 • FontAwesome 6 • Google Fonts (Inter)

↓

Serveur Web — Apache 2.4 + mod_proxy
Reverse proxy vers Gunicorn • SSL/TLS via Plesk
Static files servis directement par Apache

↓

Application — Django 4.2 + Gunicorn
2 projets : eSchoolStructure (Hub) + MonEcole (Spoke)
ReportLab (PDF) • openpyxl (Excel) • Brevo (Email)

↓

Base de Données — MySQL 8.0 / MariaDB 10.6
`countryStructure` (Hub) + `db_monecole` (Spoke)
Cross-DB VIEWs • DatabaseRouter • `managed=False`

1.2 Repositories Git

Repository	Rôle	Serveur
`evanet08/eSchoolStructure`	Hub national	`eschool-rdc.pro`
`evanet08/MonEcole`	Spoke scolaire	`monecole.pro`

1.3 Arborescence des Projets

eSchoolStructure

```
eSchoolStructure/
|-- eSchoolStructure/        # Projet Django principal
|   |-- settings.py          # Config BDD, SMTP, static
|   |-- urls.py              # Routage principal
|   |-- db_routers.py        # DatabaseRouter
|-- structure_app/           # Application principale
|   |-- models.py            # 25+ modeles (1145 lignes)
|   |-- views.py             # 80+ endpoints (2600+ lignes)
|   |-- auth_views.py        # Auth hierarchique
|   |-- templates/           # Templates Django
|   |-- static/              # CSS, JS, images
|   |-- migrations/          # 28 migrations
|-- docs/latex/              # Cette documentation
|-- manage.py
```

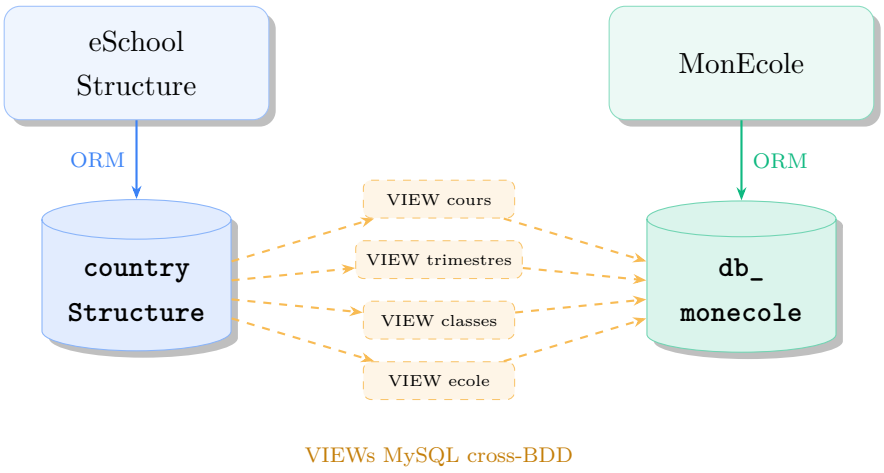
MonEcole

```
MonEcole/
|-- MonEcole/                # Projet Django principal
|   |-- settings.py
|   |-- urls.py
|   |-- db_routers.py        # CountryStructureRouter
|-- MonEcole_app/
|   |-- models/              # Modeles repartis en modules
|   |   |-- country_structure.py  # Tables Hub (managed=False)
|   |   |-- classe.py        # Classes, cycles, sections
|   |   |-- eleves/          # Eleve, Inscription, Notes
|   |   |-- evaluations/     # Notes, Deliberations
|   |   |-- recouvrement/    # Paiements, Rubriques
|   |-- views/               # Views reparties en modules
|   |   |-- direction_views/ # Dashboard directeur
|   |   |-- inscription/     # CRUD eleves
|   |   |-- evaluation/      # Notes et deliberations
|   |   |-- enseignement/    # Cours et attributions
|   |   |-- recouvrement/    # Finances
|   |-- templates/           # 60+ templates HTML
```

Chapitre 2

Architecture Multi-Base de Données

2.1 Vue d'Ensemble du Schéma Distribué



2.2 Le DatabaseRouter

Le `CountryStructureRouter` aiguille les requêtes ORM vers la bonne base :

db_routers.py

```
class CountryStructureRouter:
    """
    Route les modeles managed=False vers countryStructure,
    tout le reste vers la base par defaut (db_monecole).
    """
    hub_models = {
        'session', 'trimestre', 'periode', 'mention',
        'pays', 'classe', 'cycle',
    }

```

```
    def db_for_read(self, model, **hints):
        if model._meta.db_table in self.hub_models:
            return 'countryStructure'
        if not model._meta.managed:
            return 'countryStructure'
        return 'default'

    def db_for_write(self, model, **hints):
        if not model._meta.managed:
            return 'countryStructure'
        return 'default'

    def allow_relation(self, obj1, obj2, **hints):
        return True  # Cross-DB relations autorisees

    def allow_migrate(self, db, app_label, model_name=None, **hints):
        return db == 'default'
```

2.3 Pattern managed=False

Les tables du Hub réutilisées dans le Spoke sont déclarées avec `managed = False` :

MonEcole models/country_structure.py

```
class Trimestre(models.Model):
    """
    Table de reference : countryStructure.trimestres
    managed = False : Django ne modifie pas le schema.
    """
    id_trimestre = models.AutoField(primary_key=True)
    trimestre = models.CharField(max_length=50, unique=True)
    is_active = models.BooleanField(default=True)

    class Meta:
        db_table = 'trimestres'
        managed = False  # <-- Cle du pattern
        verbose_name = 'Trimestre'

    def __str__(self):
        return self.trimestre
```

2.4 VIEWs MySQL

2.4.1 VIEWs de Référence Simples

VIEWs 1 :1 vers le Hub

```sql
-- Trimestres
CREATE OR REPLACE VIEW `db_monecole`.`trimestres` AS
SELECT * FROM countryStructure.trimestres;

-- Sessions
CREATE OR REPLACE VIEW `db_monecole`.`sessions` AS
SELECT * FROM countryStructure.sessions;

-- Mentions
CREATE OR REPLACE VIEW `db_monecole`.`mentions` AS
SELECT * FROM countryStructure.mentions;

-- Classes
CREATE OR REPLACE VIEW `db_monecole`.`classes` AS
SELECT * FROM countryStructure.classes;
```

2.4.2 VIEW Cross-DB avec JOIN : cours_par_classe

VIEW avec jointure cross-BDD

```sql
CREATE OR REPLACE VIEW `db_monecole`.`cours_par_classe` AS
SELECT
    c.id_cours AS id_cours_classe,
    c.cours,
    c.code_cours,
    c.classe_id,
    c.domaine_id,
    ca.id_classe_active,
    ca.id_campus_id,
    ca.id_annee_id
FROM countryStructure.cours c
JOIN db_monecole.classe_active ca
  ON ca.classe_id_id = c.classe_id;
```

2.4.3 VIEW Universal Context : annee_trimestre

Piège : Le Pattern "Hardcoded Zero"

Ne JAMAIS utiliser `0 AS id_campus_id` dans une VIEW. Cela casse tous les filtres Django ORM. Toujours effectuer un JOIN pour résoudre les identifiants de contexte.

Pattern "Universal Context"

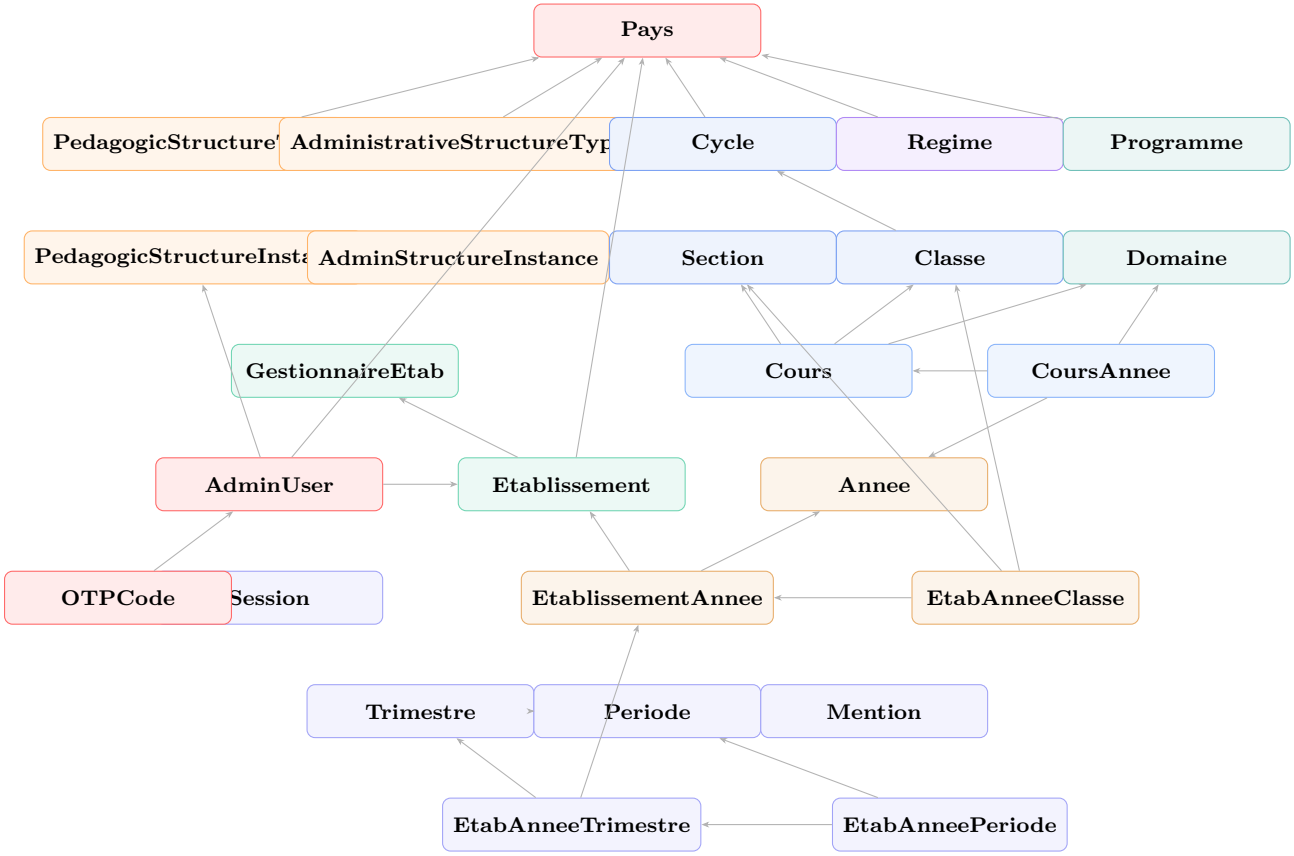
```sql
CREATE OR REPLACE VIEW `db_monecole`.`annee_trimestre` AS
SELECT
```

```sql
    t.id AS id_trimestre,
    t.trimestre_id,
    t.debut, t.fin,
    t.etat AS etat_trimestre,
    a.id_annee AS id_annee_id,
    c.id_campus AS id_campus_id,
    cca.id_cycle_actif AS id_cycle_id,
    ca.id_classe_active AS id_classe_id
FROM countryStructure.etablissements_annees_trimestres t
JOIN countryStructure.etablissements_annees ea
  ON t.etablissement_annee_id = ea.id
JOIN db_monecole.annee a
  ON a.id_annee_structure = ea.annee_id
JOIN db_monecole.campus c
  ON c.id_etablissement = ea.etablissement_id
  AND c.is_active = 1
JOIN db_monecole.classe_cycle_actif cca
  ON cca.id_campus_id = c.id_campus
  AND cca.id_annee_id = a.id_annee
  AND cca.is_active = 1
JOIN db_monecole.classe_active ca
  ON ca.cycle_id_id = cca.id_cycle_actif
  AND ca.id_campus_id = c.id_campus
  AND ca.id_annee_id = a.id_annee
  AND ca.is_active = 1;
```

Chapitre 3

Schéma de Données — eSchoolStructure

3.1 Diagramme Entité-Relation Complet



3.2 Modèles Détaillés

3.2.1 Pays — Point d'Entrée

Champ	Type	Description
`id_pays`	AutoField (PK)	Identifiant unique
`nom`	CharField(100)	Nom du pays (unique)
`sigle`	CharField(5)	Code ISO du pays (unique)
`nLevelsStructuraux`	PositiveInt	Nombre de niveaux pédagogiques
`nLevelsAdministratifs`	PositiveInt	Nombre de niveaux administratifs
`domaine`	CharField	Domaine web associé

3.2.2 Cycle — Niveaux d'Enseignement

Champ	Type	Description
`id_cycle`	AutoField (PK)	Identifiant unique
`nom`	CharField(100)	Nom (Primaire, Secondaire...)
`code`	CharField(20)	Code court (PRI, SEC...)
`duree`	PositiveInt	Durée en années
`ordre`	PositiveInt	Ordre hiérarchique
`pays`	FK → Pays	Pays d'appartenance
`hasSections`	Boolean	Possède des sections (Sci, Lit...)

3.2.3 Cours — Catalogue des Matières

Champ	Type	Description
`id_cours`	AutoField (PK)	Identifiant unique
`cours`	CharField(200)	Nom du cours (Mathématiques...)
`code_cours`	CharField(20)	Code unique (MATH01...)
`classe`	FK → Classe	Classe d'appartenance
`domaine`	FK → Domaine	Domaine par défaut
`section`	FK → Section (null)	Section (pour cycles à sections)
unique_together : (classe, section, code_cours)		

3.2.4 AdminUser — Hiérarchie de Sécurité

Champ	Type	Description
`id_admin`	AutoField (PK)	Identifiant unique
`email`	EmailField (unique)	Identifiant principal
`telephone`	CharField(20)	Numéro international
`password_hash`	CharField(128)	Hash PBKDF2-SHA256
`pays`	FK → Pays	Pays d'appartenance
`structure_instance`	FK → PedagogicInst (null)	Niveau structurel
`etablissement`	FK → Etablissement (null)	École spécifique
`email_verified`	Boolean	Email vérifié par OTP
`is_active`	Boolean	Compte actif
`created_by`	FK → self (null)	Administrateur créateur

Propriétés Dynamiques de AdminUser

- `niveau_ordre` — 0 (National), 1-N (Structure), N+1 (École)
- `niveau_nom` — "National", "Niveau X", "Établissement"
- `scope_code` — Code hiérarchique pour filtrage par préfixe
- `scope_name` — Nom de l'entité gérée
- `is_national` — True si niveau 0
- `is_validated` — True si email ou phone vérifié
- `has_password` — True si mot de passe défini

Chapitre 4

Schéma de Données — MonEcole

4.1 Vue d'Ensemble

MonEcole utilise 30+ modèles répartis dans des modules Python :

country_structure.py
Pays, Session, Trimestre,
Periode, Mention (managed=False)

classe.py
Classe, Cycle, Classe_active,
Section, Délibération

eleves/
Eleve, Inscription, Note_type,
Eleve_note, Conduite

evaluations/
Deliberation_type, Finalite,
5 types de résultats, Evaluation

recouvrement/
Paiement, Rubrique_variable,
Prix, Derogation

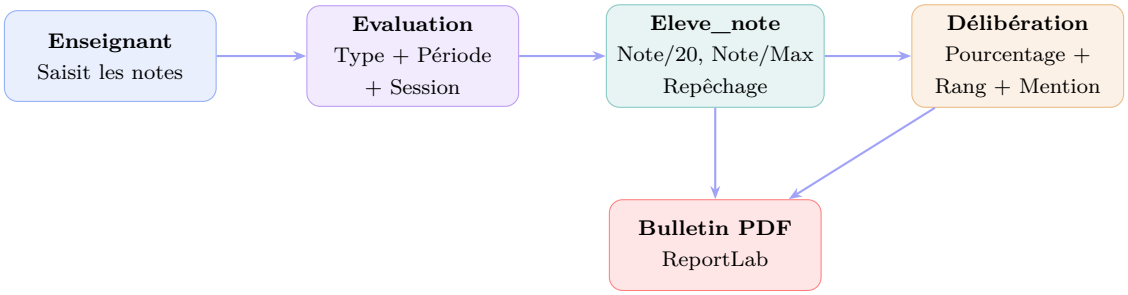
Autres
Campus, Ecole, Annee, Module,
Personnel, Horaire, Salle

4.2 Modèle Élève — Fiche Complète

Le modèle `Eleve` contient 35+ champs couvrant l'identité, la filiation, la localisation et les identifiants de sécurité :

Catégorie	Champs
Identité	nom, prenom, genre, etat_civil, date_naissance
Filiation	nom_pere, prenom_pere, nom_mere, prenom_mere, tutaire
Scolarité	code_eleve, code_annee, matricule
Contact	email, email_parent, telephone
Localisation	naissance_pays/province/commune/zone, province/commune/zone_actuelle
Sécurité	password, password_parent, code_secret_parent, code_secret_eleve
Media	imageUrl (upload vers logos/eleves/)

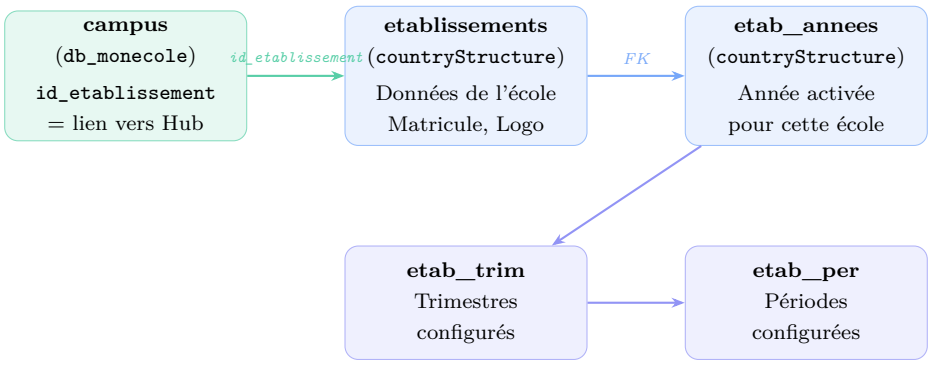
4.3 Circuit de la Note



Chapitre 5

Liaison Hub ↔ Spoke

5.1 Chaîne de Résolution du Tenant



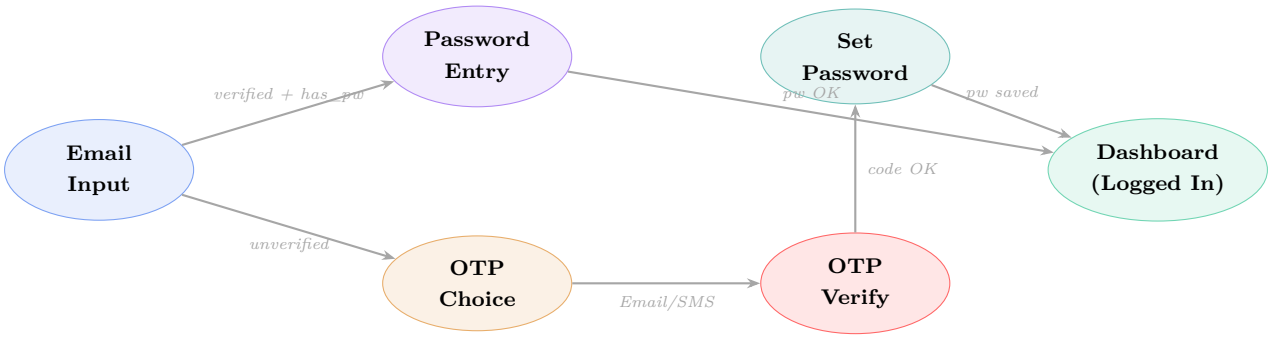
Règle d'Or : "Always Carry the Etablissement ID"

Chaque enregistrement opérationnel dans le Spoke doit avoir un chemin de résolution vers `id_etablissement` via la table `campus`. Sans ce lien, les VIEWs cross-BDD retournent des ensembles vides.

Chapitre 6

Système d'Authentification

6.1 Machine à États



6.2 Endpoints d'Authentification

Endpoint	Méthode	Description
`/api/auth/check-email/`	POST	Vérifie existence et statut
`/api/auth/request-otp/`	POST	Génère et envoie OTP
`/api/auth/verify-otp/`	POST	Valide le code OTP
`/api/auth/set-password/`	POST	Définit le mot de passe
`/api/auth/login/`	POST	Connexion classique
`/api/auth/me/`	GET	Infos utilisateur connecté
`/api/auth/logout/`	POST	Déconnexion

6.3 Décorateurs de Sécurité

Décorateurs @require_auth et @require_level

```
def require_auth(view_func):
```

```
    """Verifie que la requete provient d'un AdminUser connecte."""
    def wrapper(request, *args, **kwargs):
        user_id = request.session.get('user_id')
        if not user_id:
            return JsonResponse({'error': 'Non authentifie'}, status
                =401)
        try:
            request.admin_user = AdminUser.objects.get(id_admin=user_id)
        except AdminUser.DoesNotExist:
            request.session.flush()
            return JsonResponse({'error': 'Session invalide'}, status
                =401)
        return view_func(request, *args, **kwargs)
    return wrapper

def require_level(max_level):
    """Verifie le niveau hierarchique minimum."""
    def decorator(view_func):
        def wrapper(request, *args, **kwargs):
            if request.admin_user.niveau_ordre > max_level:
                return JsonResponse({'error': 'Acces refuse'}, status
                    =403)
            return view_func(request, *args, **kwargs)
        return wrapper
    return decorator
```

Chapitre 7

Patterns de Développement

7.1 Pattern Upsert JSON

Pattern : Upsert (Create or Update)

Un seul endpoint gère à la fois la création et la modification d'une entité. La présence ou l'absence de l'ID dans le payload détermine l'opération.

Exemple : save_cours

```
@require_http_methods(["POST"])
def save_cours(request):
    data = json.loads(request.body)
    id_cours = data.get('id_cours')  # None = create

    payload = {
        'cours': data.get('cours'),
        'code_cours': data.get('code_cours'),
        'classe': get_object_or_404(Classe, id_classe=data['id_classe'])
            ,
        'domaine': get_object_or_404(Domaine, id_domaine=data['
            id_domaine']),
    }

    if id_cours:
        # UPDATE
        instance = get_object_or_404(Cours, id_cours=id_cours)
        for key, val in payload.items():
            setattr(instance, key, val)
        instance.save()
    else:
        # CREATE
```

```
        if Cours.objects.filter(
            classe=payload['classe'],
            code_cours=payload['code_cours']
        ).exists():
            return JsonResponse({'error': 'Code duplique'}, status=400)
        instance = Cours.objects.create(**payload)

    return JsonResponse({'success': True, 'id': instance.id_cours})
```

7.2 Pattern Checkbox Activation

Pattern : Activation par Checkbox

L'utilisateur voit TOUS les éléments du catalogue. Les éléments actifs dans le contexte courant sont cochés. Cocher/décocher déclenche une insertion/suppression atomique via AJAX.

MATH01 — Mathématiques *(Pondération : 3, Heures : 6h)*

FRAN01 — Français *(Pondération : 2, Heures : 4h)*

ANGL01 — Anglais *(Non activé cette année)*

PHYS01 — Physique *(Pondération : 2, Heures : 3h)*

CHIM01 — Chimie *(Non activé cette année)*

Cocher = INSERT dans cours_annee | Décocher = DELETE (si pas de notes liées)

Chapitre 8

Déploiement et DevOps

8.1 Workflow de Déploiement

1. **Local** git add + commit → 2. **SSH** Connexion au serveur → 3. **Pull** git fetch origin git reset –hard

4. **Static** collectstatic –noinput → 5. **Migrate** manage.py migrate → 6. **Restart** systemctl restart eschool-structure

Commande de déploiement complète

```
sshpass -p 'PASSWORD' ssh root@87.106.23.108 \
  "cd /var/www/vhosts/eschool-rdc.pro/httpdocs && \
   git fetch origin && \
   git reset --hard origin/main && \
   ./venv/bin/python3 manage.py collectstatic --noinput && \
   ./venv/bin/python3 manage.py migrate --noinput && \
   systemctl restart eschool-structure"
```

8.2 Vérification Post-Déploiement

Commandes de vérification

```
# Verifier le statut HTTP
curl -s -o /dev/null -w "%{http_code}\n" https://eschool-rdc.pro/

# Verifier les logs Gunicorn
journalctl -u eschool-structure --no-pager -n 20

# Verifier le service
```

```
systemctl status eschool-structure
```

Chapitre 9

Guide du Développeur

9.1 Comment Ajouter un Nouveau Modèle

1. Définir la classe dans `models.py` avec `db_table` explicite
2. Ajouter les champs avec les contraintes (`unique_together`, FK...)
3. Exécuter `python manage.py makemigrations`
4. Vérifier la migration générée, ajuster si nécessaire (cf. migration `0028`)
5. Exécuter `python manage.py migrate`

9.2 Comment Ajouter un Endpoint API

1. Créer la vue dans `views.py` avec le décorateur `@require_http_methods`
2. Utiliser le pattern Upsert si applicable
3. Toujours retourner un `JsonResponse` avec `success: True/False`
4. Enregistrer l'URL dans `urls.py`
5. Tester avec `curl` ou depuis le navigateur

9.3 Comment Créer une Migration Personnalisée

Lorsque l'état de la base de production diffère du schéma Django attendu :

Pattern RunSQL pour migrations de production

```
from django.db import migrations

class Migration(migrations.Migration):
    dependencies = [('structure_app', '0027_previous')]

```

```
    operations = [
        migrations.RunSQL(
            sql="""
            -- Ajout conditionnel de colonne
            SET @col_exists = (
                SELECT COUNT(*)
                FROM information_schema.COLUMNS
                WHERE TABLE_SCHEMA=DATABASE()
                AND TABLE_NAME='cours'
                AND COLUMN_NAME='section_id'
            );
            SET @sql = IF(@col_exists = 0,
                'ALTER TABLE cours ADD COLUMN section_id INT NULL',
                'SELECT 1');
            PREPARE stmt FROM @sql;
            EXECUTE stmt;
            """,
            reverse_sql="ALTER TABLE cours DROP COLUMN section_id"
        ),
    ]
```

Annexe A

Variables d'Environnement

Variable	Description	Exemple
`DB_HOST`	Hôte MySQL	`localhost`
`DB_NAME`	Nom de la base	`countryStructure`
`DB_USER`	Utilisateur MySQL	`link2ictgroup`
`DB_PASSWORD`	Mot de passe MySQL	(secret)
`DB_PORT`	Port MySQL	`3306`
`EMAIL_HOST`	Serveur SMTP	`monecole.pro`
`EMAIL_PORT`	Port SMTP	`465`
`EMAIL_USE_SSL`	Utiliser SSL	`True`
`EMAIL_HOST_USER`	Email expéditeur	`noreply@monecole.pro`
`SECRET_KEY`	Clé Django	(secret)

Annexe B

Liste Complète des VIEWs MySQL

VIEW	Type	Source Hub
`classes`	1 :1	`countryStructure.classes`
`cours`	1 :1	`countryStructure.cours`
`sessions`	1 :1	`countryStructure.sessions`
`trimestres`	1 :1	`countryStructure.trimestres`
`periodes`	1 :1	`countryStructure.periodes`
`mentions`	1 :1	`countryStructure.mentions`
`classe_cycle`	1 :1	`countryStructure.cycles`
`ecole`	Filtré	`countryStructure.etablissements`
`cours_par_classe`	Cross-JOIN	`cours` × `classe_active`
`annee_trimestre`	Universal	Multi-JOIN (5 tables)
`annee_periode`	Universal	Multi-JOIN via `annee_trimestre`